

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 2**



ANDROID LAYOUT

Oleh:

Dessy Nurulita

NIM. 2310817220024

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2024**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN I
MODUL 1

Laporan Praktikum Pemrograman Mobile Modul 2: Android Layout ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Dessy Nurulita
NIM : 2310817220024

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1.....	6
A. Source Code.....	7
B. Output Program	12
C. Pembahasan	13
D. Tautan Git	22

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi.....	6
Gambar 2. Tampilan Aplikasi Setelah Dijalankan	7
Gambar 3. Screenshot Hasil Jawaban Soal 1 (Tampilan Awal Aplikasi)	12
Gambar 4. Screenshot Hasil Jawaban Soal 1 (Tampilan Toast Error)	12
Gambar 5. Screenshot Hasil Jawaban Soal 1 (Hasil Setelah Dihitung)	13
Gambar 6. Screenshot Hasil Jawaban Soal 1 (Tampilan Aplikasi Jika Layar Dirotate)	13

DAFTAR TABEL

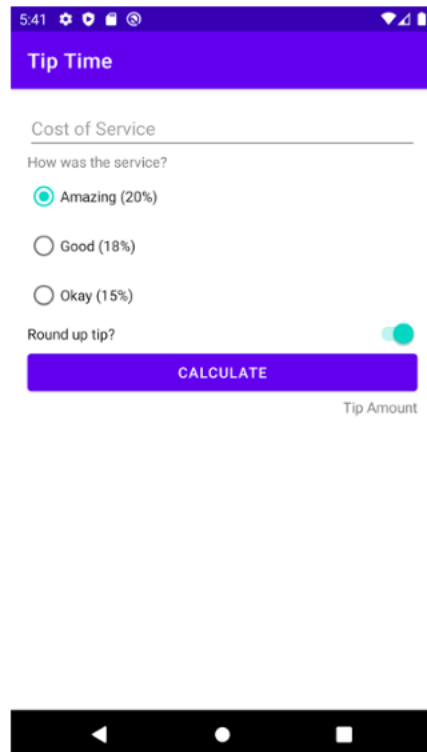
Tabel 1. Source Code Jawaban Soal 1.....	8
Tabel 2. Source Code Jawaban Soal 1.....	11
Tabel 3. Source Code Jawaban Soal 1.....	11

SOAL 1

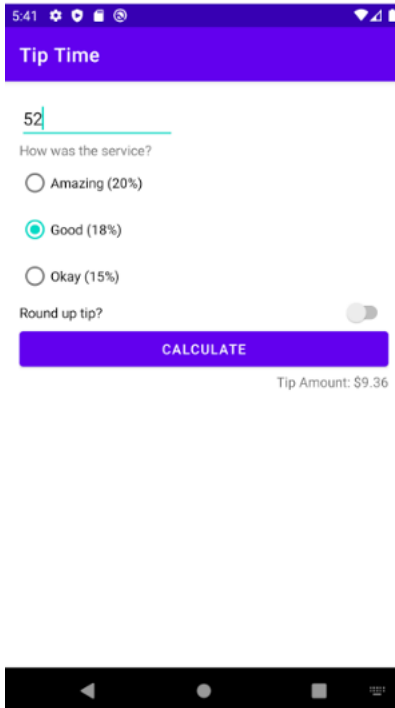
Soal Praktikum:

Buatlah sebuah aplikasi kalkulator tip yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

1. Input Biaya Layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
2. Pilihan Persentase Tip: Pengguna dapat memilih persentase tip yang diinginkan dari opsi yang disediakan, yaitu 15%, 18%, dan 20%.
3. Pengaturan Pembulatan Tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
4. Tampilan Hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 1. Tampilan Awal Aplikasi



Gambar 2. Tampilan Aplikasi Setelah Dijalankan

A. Source Code

1. MainActivity.kt

```

1 package com.example.calculatetipapp
2 package com.example.calculatetipapp
3
4 import android.os.Bundle
5 import androidx.activity.ComponentActivity
6 import androidx.activity.compose.setContent
7 import androidx.compose.material3.Surface
8 import androidx.compose.ui.Modifier
9 import androidx.compose.foundation.layout.fillMaxSize
10 import
11 com.example.calculatetipapp.ui.theme.CalculateTipAppTheme
12
13 class MainActivity : ComponentActivity() {
14     override fun onCreate(savedInstanceState: Bundle?) {
15         super.onCreate(savedInstanceState)
16         setContent {
17             CalculateTipAppTheme {
18                 Surface(
19                     modifier = Modifier.fillMaxSize()
20                 ) {
21                     TipCalculatorApp()
22                 }
23             }
24         }
25     }
26 }

```

23	}
24	}
	}
	}

Tabel 1. Source Code Jawaban Soal 1

2. TipCalculatorApp.kt

1	package com.example.calculatetipapp
2	
3	import android.widget.Toast
4	import androidx.compose.foundation.layout.*
5	import androidx.compose.foundation.rememberScrollState
6	import androidx.compose.foundation.text.KeyboardOptions
7	import androidx.compose.foundation.verticalScroll
8	import androidx.compose.material3.*
9	import androidx.compose.material3.TopAppBar
10	import androidx.compose.runtime.Composable
11	import androidx.compose.ui.Alignment
12	import androidx.compose.ui.Modifier
13	import androidx.compose.ui.graphics.Color
14	import androidx.compose.ui.platform.LocalContext
15	import androidx.compose.ui.text.font.FontWeight
16	import androidx.compose.ui.text.input.KeyboardType
17	import androidx.compose.ui.unit.dp
18	import androidx.lifecycle.viewmodel.compose.viewModel
19	
20	@OptIn(ExperimentalMaterial3Api::class)
21	@Composable
22	fun TipCalculatorApp(tipViewModel: TipViewModel = viewModel())
23	{
24	val context = LocalContext.current
25	val scrollState = rememberScrollState()
26	
27	Scaffold(
28	topBar = {
29	TopAppBar(
30	title = {
31	Text(
32	text = "Tip Time",
33	color = Color.White,
34	fontWeight = FontWeight.Bold
35	)
36	},
37	colors =
38	TopAppBarDefaults.smallTopAppBarColors(containerColor =
39	Color(0xFF6200EE))
40	)
41	}
42	) { innerPadding ->


```

43         Column(
44             modifier = Modifier
45                 .padding(innerPadding)
46                 .padding(16.dp)
47                 .fillMaxSize()
48                 .verticalScroll(scrollState) // Aktifkan
49 scroll
50         ) {
51             TextField(
52                 value = tipViewModel.serviceCostInput,
53                 onValueChange = {
54 tipViewModel.serviceCostInput = it },
55                 placeholder = {
56                     if
57 (tipViewModel.serviceCostInput.isEmpty()) {
58                         Text("Cost of Service", color =
59 Color.Gray)
60                     }
61                 },
62                 keyboardOptions =
63 KeyboardOptions.Default.copy(keyboardType =
64 KeyboardType.Number),
65                 modifier = Modifier.fillMaxWidth(),
66                 colors = TextFieldDefaults.textFieldColors(
67                     containerColor = Color.Transparent,
68                 )
69             )
70
71             Spacer(modifier = Modifier.height(16.dp))
72             Text("How was the service?", color = Color.Gray)
73
74             Spacer(modifier = Modifier.height(8.dp))
75             listOf(
76                 "Amazing (20%)" to 20,
77                 "Good (18%)" to 18,
78                 "Okay (15%)" to 15
79             ).forEach { (label, value) ->
80                 Row(verticalAlignment =
81 Alignment.CenterVertically) {
82                     RadioButton(
83                         selected =
84 tipViewModel.selectedTipPercent == value,
85                         onClick = {
86 tipViewModel.selectedTipPercent = value }
87                     )
88                     Text(text = label)
89                 }
90             }
91
92             Spacer(modifier = Modifier.height(8.dp))
93             Row(
94                 modifier = Modifier

```

```

95         .fillMaxWidth()
96         .padding(top = 16.dp),
97         verticalAlignment =
98 Alignment.CenterVertically,
99         horizontalArrangement =
100 Arrangement.SpaceBetween
101     ) {
102         Text("Round up tip?")
103         Switch(
104             checked = tipViewModel.roundUp,
105             onCheckedChange = { tipViewModel.roundUp =
106 it }
107         )
108     }
109
110     Spacer(modifier = Modifier.height(16.dp))
111     Button(
112         onClick = {
113             val cost =
114 tipViewModel.serviceCostInput.toDoubleOrNull()
115             if (cost == null || cost <= 0.0) {
116                 Toast.makeText(context, "Jangan
117 masukkan nilai 0 atau minus!", Toast.LENGTH_SHORT).show()
118                 tipViewModel.showTip = false
119             } else {
120                 tipViewModel.onCalculateClicked()
121             }
122         },
123         colors =
124 ButtonDefaults.buttonColors(containerColor =
125 Color(0xFF6200EE)),
126         modifier = Modifier.fillMaxWidth()
127     ) {
128         Text("CALCULATE")
129     }
130
131     if (!tipViewModel.showTip) {
132         Spacer(modifier = Modifier.height(8.dp))
133         Text(
            text = "Tip Amount",
            color = Color.Gray,
            style =
MaterialTheme.typography.bodySmall,
            modifier = Modifier.align(Alignment.End)
        )
    }

    if (tipViewModel.showTip) {
        Spacer(modifier = Modifier.height(24.dp))
        Text(
            text = "Tip Amount:
\\${tipViewModel.calculatedTip}",

```

	<pre> color = Color.Gray, style = MaterialTheme.typography.bodySmall, modifier = Modifier.align(Alignment.End)) } } } } </pre>
--	---

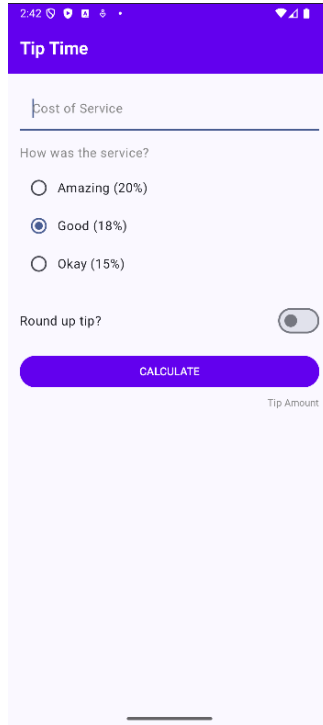
Tabel 2. Source Code Jawaban Soal 1

3. TipViewModel.kt

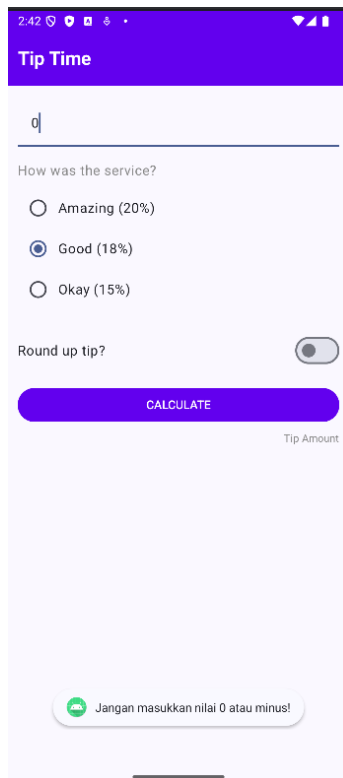
1	package com.example.calculatetipapp
2	
3	import androidx.compose.runtime.getValue
4	import androidx.compose.runtime.mutableStateOf
5	import androidx.compose.runtime.setValue
6	import androidx.lifecycle.ViewModel
7	import kotlin.math.ceil
8	
9	class TipViewModel : ViewModel() {
10	var serviceCostInput by mutableStateOf("")
11	var selectedTipPercent by mutableStateOf(18)
12	var roundUp by mutableStateOf(false)
13	var showTip by mutableStateOf(false)
14	
15	val calculatedTip: String
16	get() {
17	val cost = serviceCostInput.toDoubleOrNull() ?:
18	return "0.00"
19	var tip = cost * selectedTipPercent / 100
20	if (roundUp) tip = ceil(tip)
21	return String.format("%.2f", tip)
22	}
23	
24	fun onCalculateClicked() {
25	showTip = true
26	}
	}

Tabel 3. Source Code Jawaban Soal 1

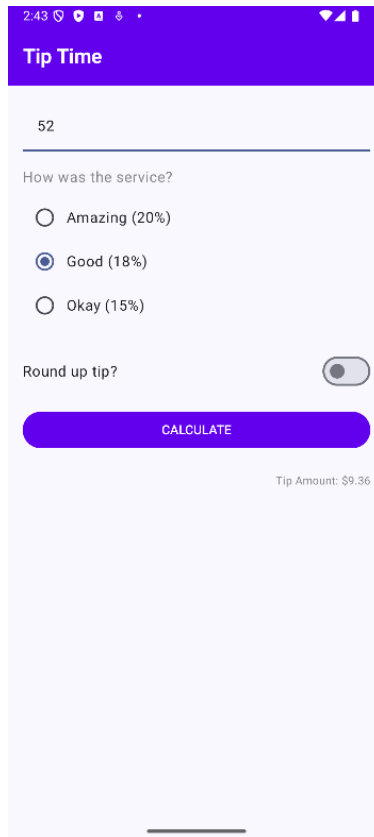
B. Output Program



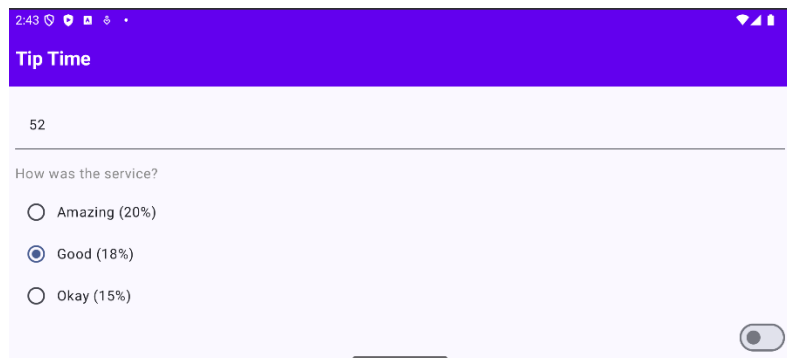
Gambar 3. Screenshot Hasil Jawaban Soal 1 (Tampilan Awal Aplikasi)



Gambar 4. Screenshot Hasil Jawaban Soal 1 (Tampilan Toast Error)



Gambar 5. Screenshot Hasil Jawaban Soal 1 (Hasil Setelah Dihitung)



Gambar 6. Screenshot Hasil Jawaban Soal 1 (Tampilan Aplikasi Jika Layar Dirotate)

C. Pembahasan

Berikut merupakan pembahasan program dari masing-masing file:

1. MainActivity.kt:

- Pada line 1, dideklarasikan nama package file Kotlin dan menunjukkan nama tempat file ini berada.
- Pada line 3, diimport `Bundle` untuk mengkomunikasikan dan menyimpan data antar aktivitas atau saat activity di-pause atau rotate.
- Pada line 4, diimport `androidx.activity.ComponentActivity` untuk kelas dasar dari activity yang support Jetpack Compose.
- Pada line 5, diimport `androidx.activity.compose.setContent` untuk menampilkan UI berbasis Jetpack Compose di dalam activity.
- Pada line 6, diimport `androidx.compose.material3.Surface` untuk untuk membungkus konten UI dengan properti seperti warna latar atau shape.
- Pada line 7, diimport `androidx.compose.ui.Modifier` untuk memodifikasi tampilan UI Compose seperti padding, ukuran, posisi, dll.
- Pada line 8, diimport `androidx.compose.foundation.layout.fillMaxSize` untuk membuat komponen memenuhi seluruh ukuran layar.
- Pada line 9, diimport `com.example.calculatetipapp.ui.theme.CalculateTipAppTheme` untuk mengimpor tema compose.
- Pada line 11, terdapat `class MainActivity : ComponentActivity()` yang mendefinisikan `MainActivity` sebagai turunan dari `ComponentActivity`, artinya ini adalah activity utama aplikasi.
- Pada line 12, terdapat override fungsi `onCreate`, yaitu titik masuk ketika activity pertama kali dibuat. `savedInstanceState` menyimpan state sebelumnya jika ada.
- Pada line 13, terdapat pemanggilan implementasi `onCreate` dari superclass agar aktivitas bisa berfungsi normal.
- Pada line 14, terdapat `setContent {` untuk memulai UI Compose. Segala yang ada di dalam kurung kurawal ini adalah UI berbasis Compose.
- Pada line 15, terdapat `CalculateTipAppTheme {` yang akan menerapkan tema khusus yang diimpor tadi ke seluruh UI di dalamnya.

- Pada line 16, terdapat `Surface (` yang akan membuat container `Surface` sebagai dasar UI.
- Pada line 17, terdapat `modifier = Modifier.fillMaxSize()` yang akan menjadikan modifier ini membuat `Surface` memenuhi seluruh ukuran layar.
- Pada line 19, akan memanggil fungsi `TipCalculatorApp()` yang merupakan composable utama yang berisi tampilan kalkulator tip.

2. **TipCalculatorApp.kt**

- Pada line 1, dideklarasikan nama package file Kotlin dan menunjukkan nama tempat file ini berada.
- Pada line 3, diimport `Toast` untuk menampilkan pesan pemberitahuan jika ada error atau ketika user salah memasukkan input.
- Pada line 4, diimport `foundation.layout` untuk mengimpor berbagai layout tools: `Column`, `Row`, `Spacer`, `padding`, `fillMaxSize`, dll.
- Pada line 5, diimport `rememberScrollState` untuk membuat scroll state yang diingat `Compose`.
- Pada line 6, diimport `KeyboardOptions` untuk mengatur opsi keyboard.
- Pada line 7, diimport `verticalScroll` agar konten di-layout bisa di-scroll ke atas-bawah.
- Pada line 8, diimport komponen `material3`, seperti `TextField`, `Button`, `Text`, `Switch`, dll.
- Pada line 9, diimport komponen `TopAppBar`.
- Pada line 10, diimport `androidx.compose.runtime.Composable` untuk mendefinisikan fungsi UI yang bisa dipakai di `Compose`.
- Pada line 11, diimport `androidx.compose.ui.Alignment` untuk mengatur posisi alignment elemen.
- Pada line 12, diimport `androidx.compose.ui.Modifier` untuk styling seperti `padding`, `fill`, dll.
- Pada line 13, diimport `Color` untuk mengatur warna.

- Pada line 14, diimport `androidx.compose.ui.platform.LocalContext` untuk mengakses konteks Android (dibutuhkan buat Toast).
- Pada line 15, diimport `text.font.FontWeight` untuk mengatur ketebalan font.
- Pada line 16, diimport `KeyboardType` untuk menentukan tipe keyboard.
- Pada line 17, diimport `unit.dp` untuk menentukan ukuran padding, spacing dalam satuan dp.
- Pada line 18, diimport `androidx.lifecycle.viewmodel.compose.viewModel` untuk memanggil ViewModel dari Compose.
- Pada line 20, terdapat `@OptIn(ExperimentalMaterial3Api::class)` untuk mengizinkan penggunaan fitur eksperimental dari Material3.
- Pada line 21, terdapat `@Composable` yang menandakan bahwa fungsi ini sebagai UI di Jetpack Compose.
- Pada line 22, terdapat `fun TipCalculatorApp(tipViewModel: TipViewModel = viewModel()) {` yang merupakan fungsi utama UI dan akan mengambil `TipViewModel` agar data state bisa dipantau.
- Pada line 23, terdapat `val context = LocalContext.current` yang akan mengambil Context aplikasi, dibutuhkan untuk Toast.
- Pada line 24, terdapat `val scrollState = rememberScrollState()` yang akan menyimpan status scroll agar bisa dipakai pada `verticalScroll`.
- Pada line 26, terdapat `Scaffold()` yang merupakan komponen layout dasar yang menyediakan `topBar`, `bottomBar`, dll.
- Pada line 27, terdapat `topBar = {` yang menentukan isi dari `topBar`.
- Pada line 28, terdapat `TopAppBar()` yang artinya akan menggunakan `TopAppBar` dari Material3.
- Pada line 29, terdapat `title = {` yang menampilkan judul di `TopBar`.

- Pada line 30 hingga 34, terdapat `Text(text = "Tip Time", color = Color.White, fontWeight = FontWeight.Bold)` yang akan menampilkan teks "Tip Time" dengan warna putih dan bold.
- Pada line 36, terdapat `colors = TopAppBarDefaults.smallTopAppBarColors(containerColor = Color(0xFF6200EE))` yang akan mengatur warna background TopAppBar menjadi warna ungu.
- Pada line 39, terdapat `{ innerPadding ->` yang merupakan parameter `innerPadding` dari Scaffold.
- Pada line 40, terdapat `Column(` yang akan mengatur isi aplikasi dalam satu kolom vertikal.
- Pada line 41 hingga 46, terdapat `modifier = Modifier.padding(innerPadding).padding(16.dp).fillMaxSize().verticalScroll(scrollState)` yang merupakan modifier untuk mengatur padding, memenuhi layar, dan bisa di-scroll.
- Pada line 47, terdapat `TextField(` yang akan membuat inputan untuk cost of service.
- Pada line 48 hingga 49, terdapat `value = tipViewModel.serviceCostInput, onChange = { tipViewModel.serviceCostInput = it },` yang akan mengambil dan menyimpan input user ke dalam ViewModel.
- Pada line 50 hingga 54, terdapat `placeholder = { if (tipViewModel.serviceCostInput.isEmpty()) { Text("Cost of Service", color = Color.Gray) } },` yang akan menampilkan placeholder jika input masih kosong. Di dalamnya ada text "Cost of Service".
- Pada line 55, terdapat `keyboardOptions = KeyboardOptions.Default.copy(keyboardType = KeyboardType.Number)` yang akan mengatur keyboard jadi angka.

- Pada line 56, terdapat `modifier = Modifier.fillMaxWidth()` yang akan mengatur layar menjadi lebar penuh.
- Pada line 57 hingga 58, terdapat `colors = TextFieldDefaults.textFieldColors( containerColor = Color.Transparent` yang akan mengatur background menjadi transparent.
- Pada line 62, terdapat `Spacer(modifier = Modifier.height(16.dp))` yang akan menampilkan teks dengan jarak 16dp.
- Pada line 63, terdapat `Text("How was the service?", color = Color.Gray)` yang akan menampilkan teks "How was the service?" dengan teks berwarna abu-abu.
- Pada line 65 terdapat `Spacer(modifier = Modifier.height(8.dp))` yang akan menampilkan teks dengan jarak 8dp.
- Pada line 66 hingga 70, terdapat `listOf("Amazing (20%)" to 20, "Good (18%)" to 18, "Okay (15%)" to 15 ).forEach { (label, value) ->` yang akan menampilkan list pilihan tip dengan bentuk radio button.
- Pada line 71, terdapat `Row(verticalAlignment = Alignment.CenterVertically) {` yang akan menampilkan baris horizontal dan memastikan isi (RadioButton dan Text) sejajar secara vertikal di tengah.
- Pada line 72 hingga 73, terdapat `RadioButton(selected = tipViewModel.selectedTipPercent == value,` yang akan menampilkan radio button.
- Pada line 74, terdapat `onClick = { tipViewModel.selectedTipPercent = value }` yang akan menampilkan nilai `selectedTipPercent` jika tombol diklik.
- Pada line 76, terdapat `Text(text = label)` yang akan menampilkan label pilihan, misalnya "Amazing (20%)", di samping radio button.

- Pada line 80, terdapat `Spacer(modifier = Modifier.height(8.dp))` yang akan memberi jarak vertikal 8dp sebelum row berikutnya.
- Pada line 81 hingga 84, terdapat `Row(modifier = Modifier.fillMaxWidth().padding(top = 16.dp)`, yang akan menampilkan baris horizontal dengan lebar penuh. dan padding atas 16dp
- Pada line 85 terdapat `verticalAlignment = Alignment.CenterVertically`, yang mengatur agar isi sejajar secara vertikal.
- Pada line 86, terdapat `horizontalArrangement = Arrangement.SpaceBetween` yang mengatur jarak antara dua elemen diatur biar satu di kiri (text), satu di kanan (switch).
- Pada line 88, terdapat `Text("Round up tip?")` yang akan menampilkan teks "Round up tip?" di sisi kiri row.
- Pada line 89 hingga 93, terdapat `Switch(checked = tipViewModel.roundUp, onChange = { tipViewModel.roundUp = it })` yang akan menampilkan toggle switch di sisi kanan, memeriksa switch menyala atau tidak tergantung nilai roundUp dari ViewModel, dan mengubah nilai roundup jika switch digeser.
- Pada line 95, terdapat `Spacer(modifier = Modifier.height(16.dp))` yang akan memberi jarak vertikal 16dp sebelum tombol muncul.
- Pada line 96 hingga 97, terdapat `Button(onClick = {` yang merupakan komponen tombol. `onClick` berisi logika ketika tombol ditekan.
- Pada line 98, terdapat `val cost = tipViewModel.serviceCostInput.toDoubleOrNull()` yang akan mengubah input biaya dari string ke Double. Jika gagal (misalnya kosong/karakter), maka hasilnya null.

- Pada line 99, terdapat `if (cost == null || cost <= 0.0) {` yang akan memeriksa input. Jika input tidak valid (kosong atau ≤ 0), maka akan menjalankan program di dalamnya.
- Pada line 100 hingga 101 terdapat `Toast.makeText(context, "Jangan masukkan nilai 0 atau minus!", Toast.LENGTH_SHORT).show()` `tipViewModel.showTip = false` yang akan menampilkan peringatan menggunakan Toast, dan menyembunyikan hasil kalkulasi.
- Pada line 102 hingga 105 terdapat `} else { tipViewModel.onCalculateClicked() }`, yang akan dijalankan jika valid, lalu akan memanggil fungsi `onCalculateClicked()` dari `ViewModel` untuk menghitung tip.
- Pada line 106, terdapat `colors = ButtonDefaults.buttonColors(containerColor = Color(0xFF6200EE))`, yang akan membuat warna tombol menjadi ungu.
- Pada line 107, terdapat `modifier = Modifier.fillMaxWidth()` yang akan mengatur agar tombol memenuhi lebar container.
- Pada line 109, terdapat `Text("CALCULATE")` agar ada teks "CALCULATE" di dalam tombol.
- Pada line 112, terdapat `if (!tipViewModel.showTip) {` yang akan memeriksa jika hasil belum ditampilkan.
- Pada line 113 hingga 120, terdapat `Spacer(modifier = Modifier.height(8.dp)) Text(text = "Tip Amount", color = Color.Gray, style = MaterialTheme.typography.bodySmall, modifier = Modifier.align(Alignment.End))` yang akan menampilkan teks "Tip Amount" sebagai label, warnanya abu-abu dan rata kanan.
- Pada line 122 hingga 123, terdapat `if (tipViewModel.showTip) { Spacer(modifier = Modifier.height(24.dp))` yang akan

dijalankan jika hasil tersedia, terdapat spasi sebesar 24dp untuk memisahkan dari tombol.

- Pada line 124 hingga 125, terdapat `Text(text = "Tip Amount: \${tipViewModel.calculatedTip}", color = Color.Gray, style = MaterialTheme.typography.bodySmall, modifier = Modifier.align(Alignment.End))` yang akan menampilkan hasil kalkulasi tip dengan format \$... dan menjadikan teksnya berwarna abu-abu, kecil, dan posisinya di kanan.

3. TipViewModul.kt

- Pada line 1, dideklarasikan nama package file Kotlin dan menunjukkan nama tempat file ini berada.
- Pada line 3, diimport `getValue` agar bisa mengambil nilai dari `mutableStateOf`.
- Pada line 4, diimport `androidx.compose.runtime.mutableStateOf` yang menjadikan suatu nilai jadi state yang bisa dipantau.
- Pada line 5, diimport `setValue` agar bisa mengubah nilai `mutableStateOf`.
- Pada line 6, diimport `ViewModel`, class dari Android Jetpack yang menyimpan data UI dan survive saat rotasi layar atau perubahan konfigurasi.
- Pada line 7, diimport fungsi `ceil()` dari Kotlin, untuk pembulatan ke atas.
- Pada line 9, akan membuat kelas `TipViewModel` yang mewarisi dari `ViewModel`. yang akan menyimpan dan mengatur data yang dibutuhkan UI (hasil tip ataupun input user)
- Pada line 10, terdapat `var serviceCostInput by mutableStateOf("")` yang merupakan state untuk menyimpan input biaya layanan dari user. Awalnya string kosong ("").
- Pada line 11, terdapat `var selectedTipPercent by mutableStateOf(18)` yang akan menyimpan persentase tip yang dipilih user dengan default-nya 18%.

- Pada line 12, terdapat `var roundUp by mutableStateOf(false)` yang akan menyimpan apakah tip akan dibulatkan ke atas, default-nya false.
- Pada line 13, terdapat `var showTip by mutableStateOf(false)` yang akan menyimpan status apakah hasil tip akan ditampilkan atau belum, default-nya false.
- Pada line 15, terdapat `val calculatedTip: String` yang akan menyimpan hasil kalkulasi tip dalam bentuk string.
- Pada line 16, terdapat `get() {}` yang merupakan awal dari custom getter. Setiap kali `calculatedTip` dipanggil, logika di dalamnya ini akan dijalankan.
- Pada line 17, terdapat `val cost = serviceCostInput.toDoubleOrNull() ?: return "0.00"` yang akan mengubah input string jadi Double. Jika gagal (misal user input huruf), akan mengembalikan "0.00".
- Pada line 18, terdapat `var tip = cost * selectedTipPercent / 100` yang merupakan rumus dalam menghitung tip.
- Pada line 19, terdapat `if (roundUp) tip = ceil(tip)` yang akan diterapkan jika user memilih tip dibulatkan ke atas.
- Pada line 20, terdapat `return String.format("%.2f", tip)` yang akan mengembalikan hasil tip dalam format dua angka decimal.
- Pada line 23, terdapat `fun onCalculateClicked() {}` yang akan dipanggil Ketika tombol "Calculate" diklik.
- Pada line 24, akan set `showTip` jadi true, sehingga UI akan tahu harus menampilkan hasil tip.

D. Tautan Git

[PEMROGRAMAN-MOBILE/MODUL 2/MODUL 2 at main · desyyn/PEMROGRAMAN-MOBILE](https://github.com/desyyn/PEMROGRAMAN-MOBILE)